

**HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN I**



**Báo báo bài tập lớn
Các hệ thống phân tán
Trình soạn thảo tài liệu cộng tác thời gian thực
(collab-editor)**

**Họ và tên sinh viên : Lê Trung Phúc – B22DCCN627
: Phó Đức Phương – B22DCCN639**

Nhóm : 7

Giảng viên hướng dẫn : Kim Ngọc Bách

MỤC LỤC

I. KIẾN TRÚC HỆ THỐNG	4
1.1. Tổng quan và mục tiêu thiết kế	4
1.2. Sơ đồ kiến trúc tổng thể.....	4
1.3. Tầng Client (Frontend)	5
1.4. Tầng Server — thiết kế hai server song song.....	6
1.5. Tầng lưu trữ	7
1.6. Mô hình phân quyền và bảo mật	7
1.7. Luồng xác thực và thiết lập kết nối WebSocket.....	8
II. MÔ HÌNH ĐỒNG BỘ DỮ LIỆU.....	10
2.1. Bài toán đồng bộ trong hệ phân tán.....	10
2.2. CRDT và lý do chọn Yjs	10
2.3. Nguyên lý CRDT qua ví dụ.....	10
2.4. Giao thức đồng bộ	11
2.5. Trình tự đồng bộ thời gian thực đầu–cuối.....	12
2.6. Lưu trữ và khôi phục trạng thái	13
2.6.1. Luồng đọc (khi mở tài liệu).....	13
2.6.2. Luồng ghi (khi chỉnh sửa và khi đóng)	13
2.7. Đồng bộ ngoại tuyến với IndexedDB.....	14
2.8. Tự động tạo phiên bản (auto-versioning)	14
2.9. Tính nhất quán cuối và các đảm bảo của hệ thống.....	15
III. GIAO THỨC TRAO ĐỔI DỮ LIỆU	16
3.1. Giao thức HTTP kết hợp RESTful API.....	16
3.1.1. Giới thiệu và định nghĩa:	16
3.1.2 Cách thức áp dụng trong dự án:.....	16
3.1.3. Định dạng dữ liệu và bảo mật:	16
3.1.4. Lý do lựa chọn sử dụng:	17
3.1.5. Đánh giá ưu và nhược điểm:	17
3.2. Giao thức Websockets	17
3.2.1. Giới thiệu và định nghĩa:	17

3.2.2. Cách thức áp dụng trong dự án:.....	17
3.2.3. Định dạng dữ liệu và bảo mật:	17
3.2.4. Lý do lựa chọn sử dụng:	17
3.2.5. Đánh giá ưu và nhược điểm:	18
3.3 Giao thức Yjs Sync Protocol	18
3.3.1. Giới thiệu và định nghĩa:	18
3.3.2. Cách thức áp dụng trong dự án:.....	18
3.3.3. Định dạng dữ liệu và bảo mật:	18
3.3.4. Lý do lựa chọn sử dụng:	18
3.3.5. Đánh giá ưu và nhược điểm:	19
3.4 Giao thức Yjs Awareness Protocol.....	19
3.4.1. Giới thiệu và định nghĩa:	19
3.4.2. Cách thức áp dụng trong dự án:.....	19
3.4.3. Định dạng dữ liệu và bảo mật:	19
3.4.4. Lý do lựa chọn sử dụng:	19
3.4.5. Đánh giá ưu và nhược điểm:	20
IV. THỬ NGHIỆM VÀ ĐÁNH GIÁ	21
4.1. Giao diện trang đăng nhập, đăng kí.....	21
4.2. Giao diện trang chủ.....	23
4.3.Tạo và quản lí tài liệu	24
4.4. Chia sẻ tài liệu	26
4.5. Quản lý người dùng đang chỉnh sửa.....	27
4.6. Lịch sử chỉnh sửa và phiên bản	28
4.7. Undo/redo trong môi trường cộng tác	32

I. KIẾN TRÚC HỆ THỐNG

1.1. Tổng quan và mục tiêu thiết kế

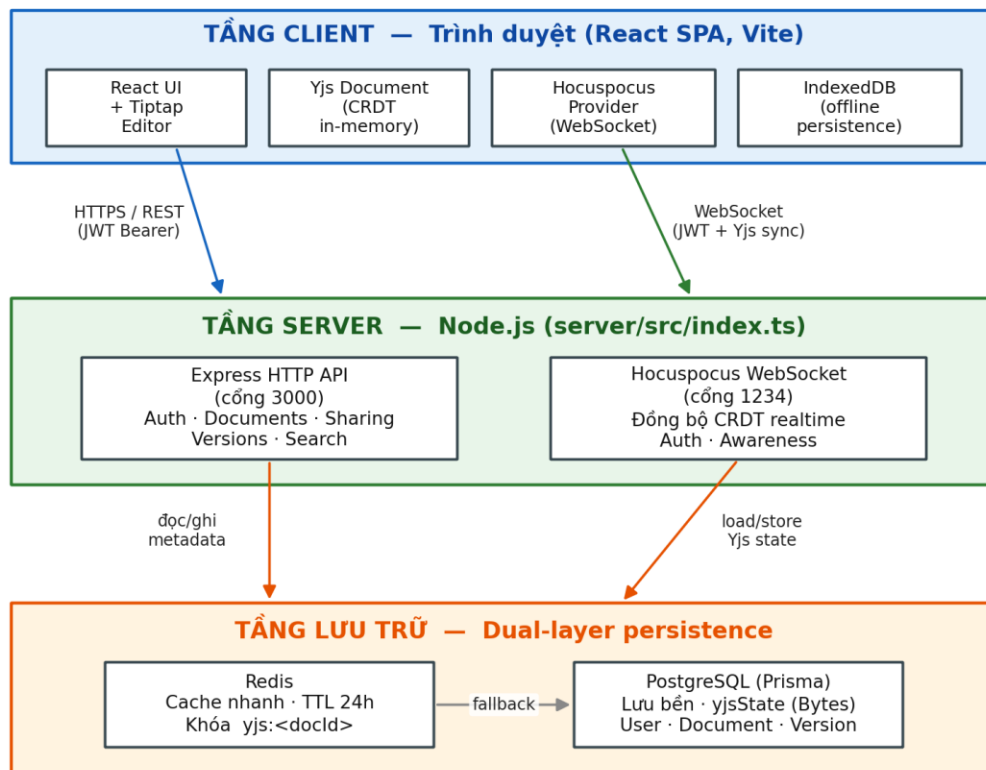
collab-editor là một trình soạn thảo tài liệu cho phép nhiều người dùng cùng chỉnh sửa một văn bản trong thời gian thực. Vì nhiều người có thể thao tác đồng thời trên cùng một tài liệu từ nhiều máy khác nhau, hệ thống mang đầy đủ đặc trưng của một hệ phân tán: nhiều bản sao dữ liệu cùng tồn tại, độ trễ mạng giữa các nút là không thể tránh khỏi, và kết nối có thể gián đoạn bất cứ lúc nào.

Để giải quyết các thách thức đó, kiến trúc được thiết kế hướng tới bốn mục tiêu cốt lõi:

- **Thời gian thực (real-time):** thay đổi của một người dùng phải hiển thị trên màn hình của những người còn lại gần như tức thì.
- **Không xung đột (conflict-free):** các chỉnh sửa đồng thời phải tự động hòa giải mà không mất dữ liệu và không cần khóa (lock).
- **Bền vững (durable):** nội dung tài liệu không được mất khi server khởi động lại hay khi mọi người dùng thoát ra.
- **Hoạt động ngoại tuyến (offline-capable):** người dùng vẫn soạn thảo được khi mất mạng và dữ liệu sẽ tự đồng bộ lại khi kết nối phục hồi.

Hệ thống được tổ chức thành ba tầng rõ rệt — tầng Client (trình duyệt), tầng Server (Node.js) và tầng Lưu trữ (Redis + PostgreSQL) — như minh họa ở Hình 1.

1.2. Sơ đồ kiến trúc tổng thể



Hình 1. Kiến trúc ba tầng của hệ thống collab-editor

Điểm đặc biệt của kiến trúc là client kết nối song song tới hai server khác nhau: một kết nối HTTP/REST tới Express dùng cho thao tác lấy dữ liệu và biến đổi (mutation) không thường xuyên (đăng nhập, tạo/xóa tài liệu, chia sẻ), và một kết nối WebSocket tới Hocuspocus dùng riêng cho luồng đồng bộ CRDT liên tục. Cách tách này được phân tích chi tiết ở mục 1.4.

1.3. Tầng Client (Frontend)

Client là một ứng dụng single-page (SPA) viết bằng React, đóng gói bằng Vite. Toàn bộ logic cộng tác được gói trong hook tùy biến useCollabEditor (client/src/hooks/useCollabEditor.ts). Hook này khởi tạo và liên kết các thành phần sau:

- **Tiptap Editor:** lớp giao diện soạn thảo (dựa trên ProseMirror), cung cấp các tính năng định dạng văn bản (đậm, nghiêng, heading, danh sách, hình ảnh, liên kết...). Đáng chú ý, StarterKit được cấu hình tắt history vì lịch sử undo/redo do Yjs đảm nhiệm.

- **Yjs Document (Y.Doc):** cấu trúc dữ liệu CRDT trong bộ nhớ, là 'nguồn sự thật' của nội dung tài liệu ở phía client. Mọi thao tác gõ phím được Tiptap chuyển thành thao tác CRDT trên Y.Doc.
- **HocuspocusProvider:** cầu nối WebSocket giữa Y.Doc và Hocuspocus server. Lưu ý quan trọng: đừng ấn dùng HocuspocusProvider chứ không dùng y-websocket, vì Hocuspocus 2.x đã hợp nhiều tài liệu trên một kết nối và mỗi message được gắn tiền tố tên tài liệu — phải dùng đúng provider để khớp giao thức.
- **IndexeddbPersistence:** lưu bản sao Y.Doc xuống IndexedDB của trình duyệt, giúp ứng dụng hoạt động ngoại tuyến và tự đồng bộ khi online trở lại.
- **CollaborationCursor (Awareness):** hiển thị con trỏ và vùng chọn của những người dùng khác theo thời gian thực, mỗi người một màu.

Đoạn mã sau (rút gọn từ useCollabEditor.ts) cho thấy cách ba thành phần Y.Doc, provider và IndexedDB được kết nối với nhau:

```
const ydoc = useMemo(() => new Y.Doc(), [documentId])

const provider = useMemo(() => new HocuspocusProvider({
  url: WS_URL, name: documentId, document: ydoc,
  token: token || '', broadcast: false,
}), [documentId, ydoc, token])

// Lưu offline; tự đồng bộ lại khi reconnect
const indexeddbProvider = useMemo(
  () => new IndexeddbPersistence(`collab-${documentId}`, ydoc),
  [documentId, ydoc])
```

1.4. Tầng Server — thiết kế hai server song song

Backend (server/src/index.ts) khởi chạy đồng thời hai server độc lập trong cùng một tiến trình Node.js:

1. **Express HTTP server (cổng 3000)** — cung cấp REST API cho xác thực (/api/auth) và quản lý tài liệu (/api/documents): liệt kê, tạo, sửa metadata, xóa, tìm kiếm, đánh dấu sao, lịch sử xem, quản lý thành viên và quản lý phiên bản (versions).
2. **Hocuspocus WebSocket server (cổng 1234)** — chuyên trách đồng bộ trạng thái CRDT thời gian thực giữa các client.

Lý do tách hai server: hai loại lưu lượng có đặc tính hoàn toàn khác nhau. REST là các yêu cầu rời rạc, request–response ngắn, hợp với mô hình HTTP không trạng thái. Ngược lại, đồng bộ cộng tác là luồng message hai chiều, tần suất cao, đòi hỏi kết nối duy trì lâu dài — đặc trưng của WebSocket. Tách biệt giúp mỗi server tối ưu cho mô hình giao tiếp của mình, đồng thời cô lập tải: việc một tài liệu có nhiều người cùng gõ liên tục không làm nghẽn các API quản lý.

```
// server/src/index.ts
app.use('/api/auth', authRouter)
app.use('/api/documents', documentRouter)
app.listen(PORT, () => ...) // HTTP 3000

hocuspocusServer.listen(WS_PORT, ...) // WebSocket 1234
```

1.5. Tầng lưu trữ

Tầng lưu trữ kết hợp hai hệ quản trị dữ liệu với vai trò bổ trợ nhau (chi tiết cơ chế ở mục 2.6):

- **Redis (cache nhanh, TTL 24 giờ):** lưu trạng thái Yjs nhị phân dưới khóa `yjs:<documentId>`, phục vụ đọc/ghi tốc độ cao trong lúc tài liệu đang được mở.
- **PostgreSQL (lưu trữ bền vững):** lưu trạng thái Yjs trong cột nhị phân `Document.yjsState`, cùng toàn bộ metadata quan hệ (người dùng, tài liệu, thành viên, phiên bản).

Lược đồ dữ liệu được mô tả bằng Prisma (`prisma/schema.prisma`) gồm các thực thể chính:

Bảng	Vai trò
User	Tài khoản người dùng (email, mật khẩu băm, tên).
Document	Tài liệu: tiêu đề, ownerId, yjsState (nhị phân), contentPreview (text để tìm kiếm), publicRole (quyền qua link công khai).
DocumentMember	Bảng nối tài liệu–người dùng kèm vai trò (OWNER/EDITOR/VIEWER); ràng buộc duy nhất (documentId, userId).
DocumentVersion	Các bản snapshot Yjs theo thời điểm; isAuto phân biệt bản tự động và bản lưu thủ công.
StarredDocument / DocumentViewed	Đánh dấu sao và lịch sử xem của từng người dùng.

1.6. Mô hình phân quyền và bảo mật

Hệ thống áp dụng kiểm soát truy cập dựa trên vai trò (RBAC) với ba vai trò: OWNER (chủ sở hữu), EDITOR (chỉnh sửa) và VIEWER (chỉ xem). Hàm `getDocumentRole` (`server/src/utils/documentAccess.ts`) xác định vai trò của một người dùng đối với một tài liệu theo thứ tự ưu tiên:

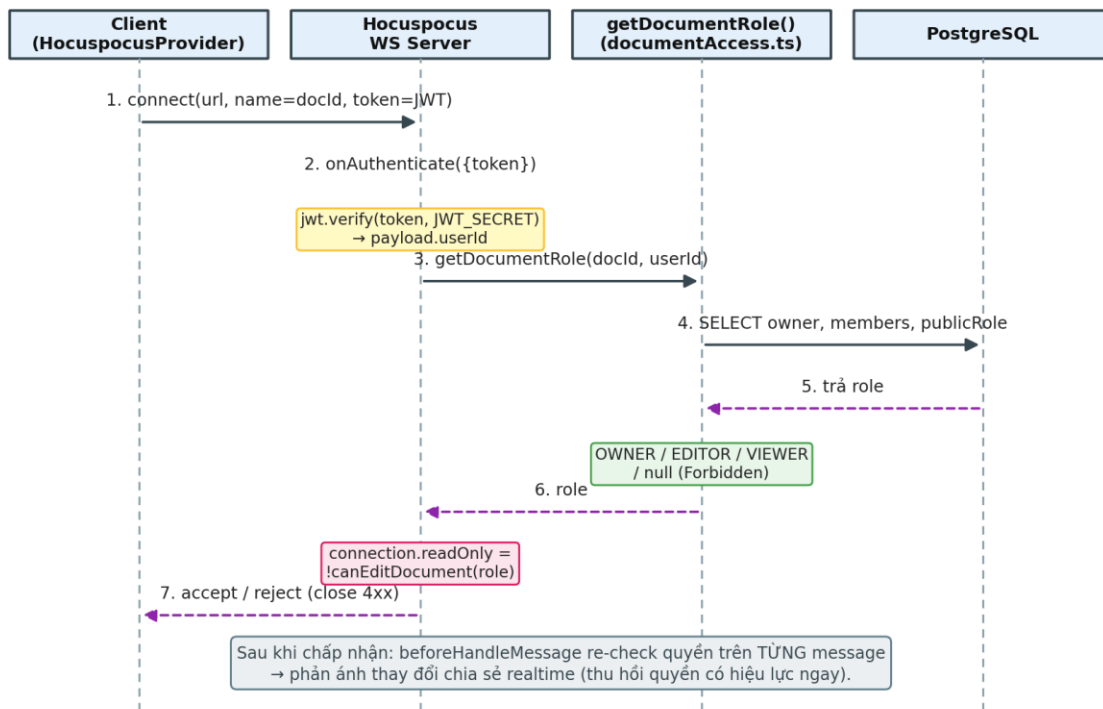
1. Nếu là chủ sở hữu (`ownerId` trùng) → OWNER.
2. Nếu là thành viên trong `DocumentMember` → vai trò tương ứng.
3. Nếu tài liệu được chia sẻ công khai (`publicRole` là VIEWER/EDITOR) → vai trò công khai đó.

4. Ngược lại → null (không có quyền, bị từ chối).

Xác thực dùng JWT. Token được ký bằng JWT_SECRET; client gửi kèm khi gọi REST (HTTP Authorization) và khi mở kết nối WebSocket. Hàm canEditDocument(role) trả về true với OWNER và EDITOR, dùng để quyết định một kết nối WebSocket là đọc–ghi hay chỉ–đọc (readOnly).

1.7. Luồng xác thực và thiết lập kết nối WebSocket

Trước khi cho phép một client tham gia chỉnh sửa, Hocuspocus thực hiện xác thực qua hook onAuthenticate (server/src/collab/hocuspocus.ts). Trình tự được mô tả ở Hình 2.



Hình 2. Trình tự xác thực và phân quyền khi mở kết nối WebSocket

```
async onAuthenticate({ token, documentName, connection }) {
  if (!token) throw new Error('Missing token')
  const payload = jwt.verify(token, JWT_SECRET) // -> userId
  const role = await getDocumentRole(documentName, payload.userId)
  if (!role) throw new Error('Forbidden')
  // VIEWER -> readOnly; Hocuspocus tự từ chối message sync
  connection.readOnly = !canEditDocument(role)
  return { userId: payload.userId, documentId: documentName, role }
}
```

Điểm tinh tế: ngoài việc kiểm tra một lần khi kết nối, hook beforeHandleMessage còn tái kiểm tra quyền trên từng message. Nhờ vậy, nếu chủ sở hữu thu hồi quyền chỉnh sửa của

một người trong khi họ đang mở tài liệu, thay đổi có hiệu lực gần như ngay lập tức mà không cần người đó tải lại trang.

II. MÔ HÌNH ĐỒNG BỘ DỮ LIỆU

2.1. Bài toán đồng bộ trong hệ phân tán

Khi nhiều người cùng chỉnh sửa một tài liệu, mỗi client giữ một bản sao cục bộ và thao tác trên đó để giao diện phản hồi tức thì (không chờ server). Hệ quả là tại một thời điểm, các bản sao có thể khác nhau. Bài toán đặt ra: làm sao để sau khi trao đổi các thay đổi, tất cả bản sao hội tụ về cùng một trạng thái, bất kể thứ tự đến của message và bất kể độ trễ mạng?

Xét một ví dụ kinh điển. Tài liệu ban đầu là chuỗi "AC". Người dùng A chèn ký tự 'B' vào giữa, gần như đồng thời người dùng B chèn 'D' vào cuối. Nếu chỉ dùng vị trí số (index) để mô tả thao tác, khi áp dụng chéo các thao tác này lên nhau, chỉ số có thể bị lệch và hai bên cho ra kết quả khác nhau (ví dụ "ABCD" so với "ABDC"). Đây chính là bài toán cần một mô hình đồng bộ đủ mạnh để giải quyết.

2.2. CRDT và lý do chọn Yjs

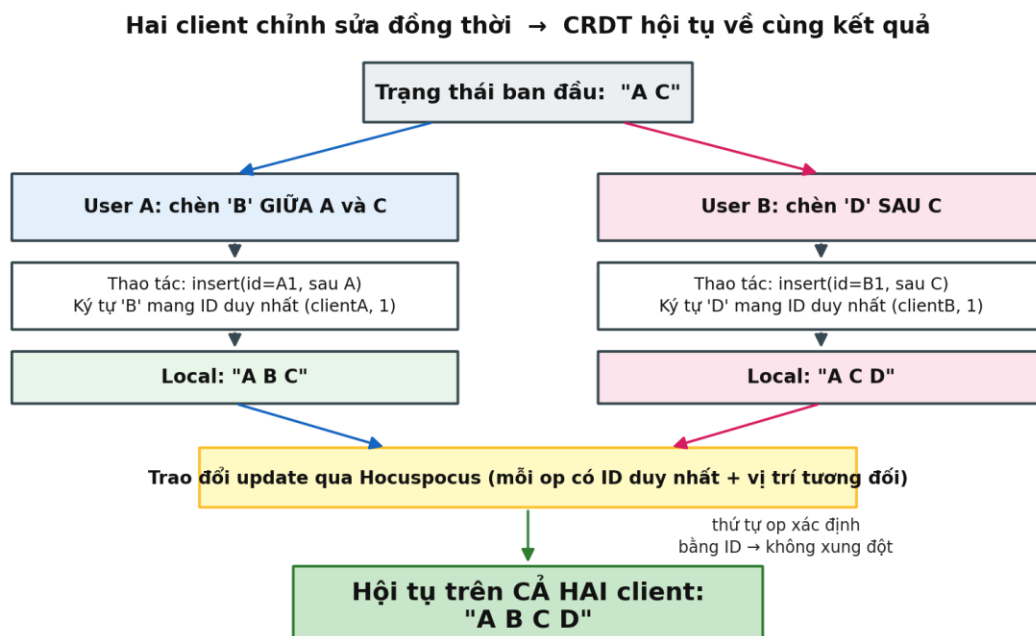
Có hai họ giải pháp phổ biến cho soạn thảo cộng tác: Operational Transformation (OT) và Conflict-free Replicated Data Types (CRDT).

- **OT** biến đổi các thao tác theo nhau để bù trừ độ lệch chỉ số. OT mạnh nhưng phức tạp và thường cần một server trung tâm đóng vai trò 'trọng tài' sắp thứ tự thao tác — khó kiểm thử và dễ sinh lỗi biên.
- **CRDT** thiết kế cấu trúc dữ liệu sao cho phép hợp nhất (merge) có tính giao hoán, kết hợp và lũy đẳng. Nhờ đó, áp dụng các thao tác theo bất kỳ thứ tự nào cũng cho cùng kết quả, không cần trọng tài trung tâm.

Dự án chọn Yjs — một thư viện CRDT hiệu năng cao. Yjs gán cho mỗi ký tự một định danh duy nhất gồm (clientID, đồng hồ logic) và lưu vị trí theo quan hệ tương đối giữa các ký tự thay vì chỉ số tuyệt đối. Chính nhờ định danh duy nhất và vị trí tương đối mà các thao tác chèn/xóa đồng thời luôn hợp nhất về một trật tự xác định, giải quyết triệt để bài toán ở mục 2.1. Yjs cũng tích hợp sẵn với Tiptap và Hocuspocus, đồng thời mã hóa cập nhật ở dạng nhị phân nhỏ gọn.

2.3. Nguyên lý CRDT qua ví dụ

Quay lại ví dụ "AC". Với Yjs, khi A chèn 'B' giữa A và C, thao tác không nói 'chèn vào vị trí số 1' mà nói 'chèn một ký tự mới có ID (clientA, 1) nằm SAU ký tự A'. Tương tự, B chèn 'D' có ID (clientB, 1) nằm SAU ký tự C. Vì mỗi ký tự neo vào một ký tự láng giềng cụ thể (chứ không vào một chỉ số có thể dịch chuyển), việc áp dụng hai thao tác này theo thứ tự nào cũng cho cùng kết quả "ABCD". Hình 3 minh họa quá trình hội tụ.



Hình 3. Hai thao tác chèn đồng thời hội tụ về cùng kết quả nhờ CRDT

Trường hợp hai người chèn ký tự vào CÙNG một vị trí (cùng neo sau một ký tự), Yjs dùng so sánh clientID để quyết định trật tự một cách tất định. Quy tắc này giống nhau trên mọi bản sao, nên kết quả vẫn nhất quán tuyệt đối. Đây là điểm mấu chốt giúp loại bỏ hoàn toàn xung đột mà không cần khóa.

2.4. Giao thức đồng bộ

Việc trao đổi trạng thái giữa client và server tuân theo giao thức của Yjs (y-protocols) mà Hocuspocus hiện thực, gồm các loại message chính:

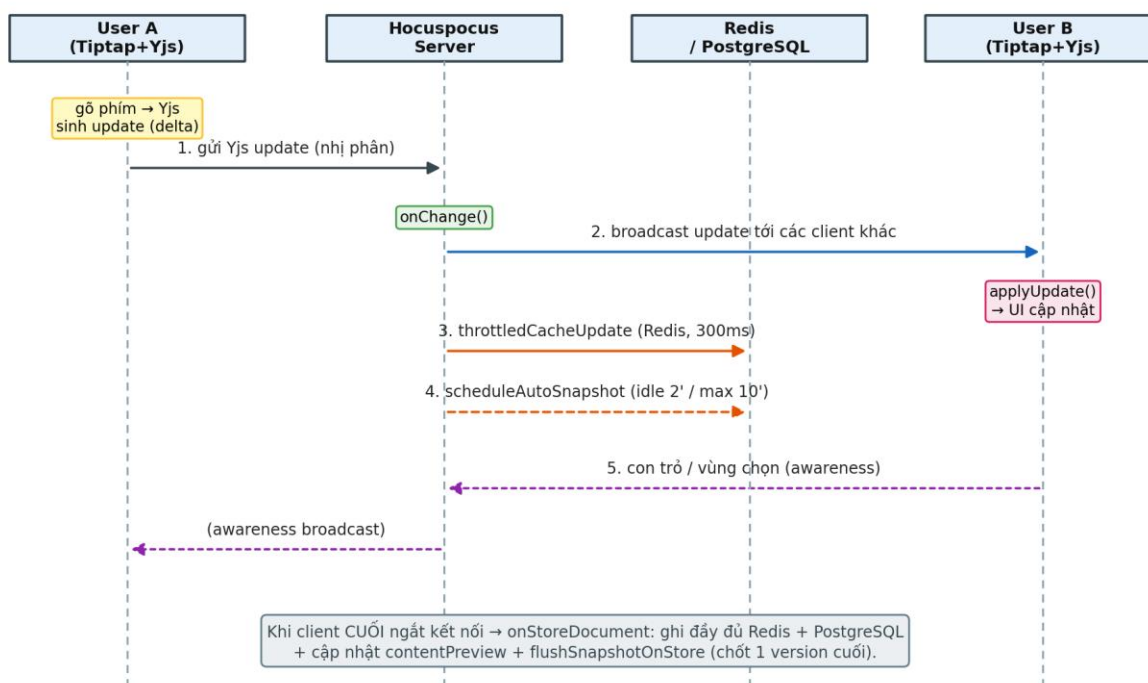
- **Sync Step 1:** khi mở tài liệu, client gửi 'state vector' (tóm tắt những gì nó đã biết) để hai bên xác định phần còn thiếu của nhau.
- **Sync Step 2 / Update:** mỗi bên gửi đúng phần thay đổi (delta nhị phân) mà bên kia chưa có. Sau đó, mọi chỉnh sửa tiếp theo được phát đi như các update tăng trưởng, rất nhỏ gọn.
- **Awareness:** một kênh riêng truyền thông tin tạm thời (con trỏ, vùng chọn, danh tính người đang online), không lưu vào tài liệu.

Ở phía client, toàn bộ phần phức tạp này được HocuspocusProvider lo liệu; lập trình viên chỉ cần truyền Y.Doc và token như đã thấy ở mục 1.3. Tham số broadcast: false tắt cơ chế

đồng bộ giữa các tab cùng trình duyệt qua BroadcastChannel, để mọi đồng bộ đi qua server một cách nhất quán.

2.5. Trình tự đồng bộ thời gian thực đầu–cuối

Hình 4 mô tả đầy đủ một vòng đồng bộ: từ lúc người dùng A gõ phím cho tới khi thay đổi hiển thị trên màn hình người dùng B và được lưu trữ.



Hình 4. Trình tự đồng bộ thời gian thực và lưu trữ phía server

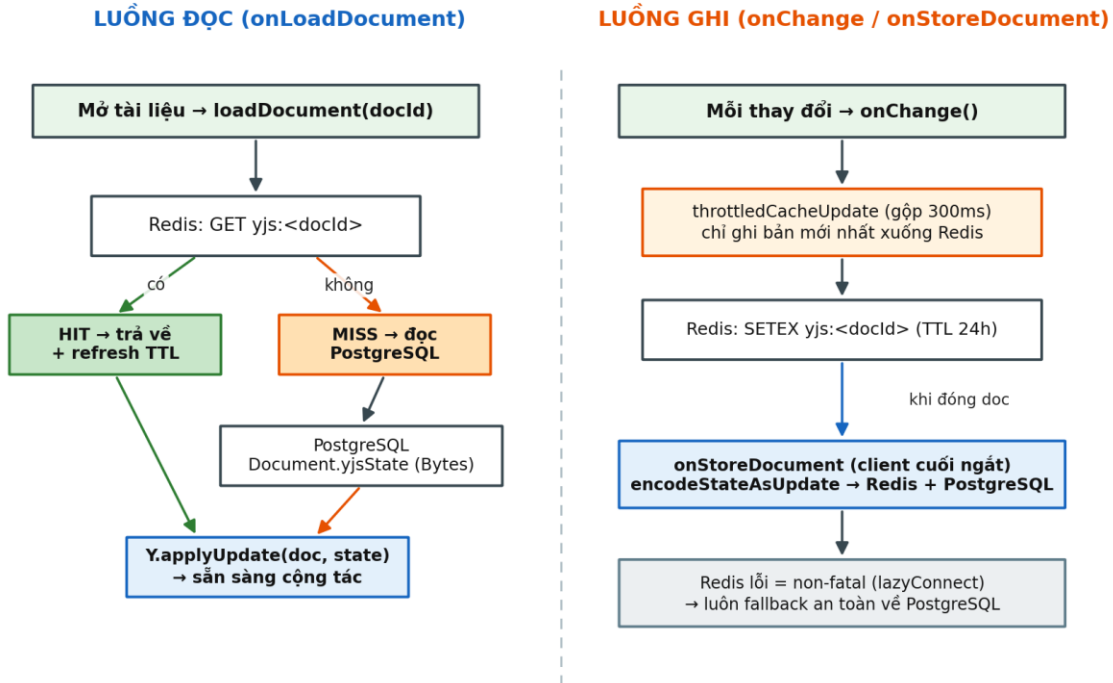
Trái tim của luồng này là hook `onChange` phía server (`server/src/collab/hocuspocus.ts`). Mỗi khi tài liệu thay đổi, server vừa phát update cho các client khác (Hocuspocus tự động làm), vừa kích hoạt hai tác vụ lưu trữ bất đồng bộ:

```
async onChange({ documentName, document, context }) {
  const state = Buffer.from(Y.encodeStateAsUpdate(document))
  // Gộp keystroke trong 300ms -> 1 lần ghi Redis
  throttledCacheUpdate(documentName, state)
  if (context.userId) {
    scheduleAutoSnapshot(documentName, context.userId, state)
    scheduleContentPreviewUpdate(documentName, extractPlainText(document))
  }
}
```

Như vậy đường đi 'nóng' (phát update cho người khác) được tách khỏi đường đi 'lưu trữ' (ghi cache, tạo phiên bản, cập nhật preview), giúp độ trễ cảm nhận của người dùng luôn thấp.

2.6. Lưu trữ và khôi phục trạng thái

Cơ chế lưu trữ hai tầng (Hình 5) được hiện thực trong server/src/collab/persistence.ts, gồm một luồng đọc và một luồng ghi.



Hình 5. Luồng đọc và ghi của lưu trữ hai tầng Redis + PostgreSQL

2.6.1. Luồng đọc (khi mở tài liệu)

Hook onLoadDocument gọi loadDocument: ưu tiên đọc Redis (nhanh); nếu trúng cache thì làm mới TTL để tài liệu đang mở liên tục không bị xóa sau 24 giờ. Nếu Redis trượt (hoặc Redis sự cố), hệ thống fallback an toàn về PostgreSQL.

```
export async function loadDocument(documentId) {
  try {
    // 1) thử Redis
    const cached = await redis.getBuffer(redisKey(documentId))
    if (cached) { redis.expire(...) // refresh TTL
      return cached }
  } catch { /* Redis lỗi -> bỏ qua, fallback DB */ }
  const doc = await prisma.document.findUnique(...) // 2) PostgreSQL
  return doc?.yjsState ? Buffer.from(doc.yjsState) : null
}
```

2.6.2. Luồng ghi (khi chỉnh sửa và khi đóng)

Trong lúc soạn thảo, throttledCacheUpdate gộp nhiều keystroke trong cửa sổ 300ms thành một lần ghi Redis duy nhất (chỉ ghi bản mới nhất), tránh việc nện Redis trên từng phím gõ nhưng vẫn đảm bảo cache luôn giữ trạng thái mới nhất.

```
export function throttledCacheUpdate(documentId, state) {
  pendingCacheState.set(documentId, state)
  if (cacheTimers.has(documentId)) return // đã có timer -> chờ
  const timer = setTimeout(async () => {
    const latest = pendingCacheState.get(documentId) // bản mới nhất
    await redis.setex(redisKey(documentId), REDIS_TTL, latest)
  }, CACHE_THROTTLE_MS) // 300ms
  cacheTimers.set(documentId, timer)
}
```

Khi người dùng CUỐI CÙNG rời tài liệu, Hocuspocus gọi `onStoreDocument` để ghi trạng thái đầy đủ xuống cả Redis và PostgreSQL (đảm bảo bền vững), đồng thời cập nhật `contentPreview` (text thuần phục vụ tìm kiếm nội dung) và chốt một phiên bản cuối. Triết lý chung: Redis là tầng tăng tốc và có thể lỗi (`lazyConnect`, mọi lỗi đều non-fatal), còn PostgreSQL là nguồn dữ liệu bền vững cuối cùng.

2.7. Đồng bộ ngoại tuyến với IndexedDB

Mỗi tài liệu có một bản sao `Y.Doc` lưu trong IndexedDB của trình duyệt qua `IndexeddbPersistence`. Nhờ đó, khi mất mạng, người dùng vẫn tiếp tục soạn thảo bình thường — các thay đổi được ghi vào CRDT cục bộ và IndexedDB. Khi kết nối phục hồi, `HocuspocusProvider` tự thực hiện lại quy trình Sync Step 1/2: trao đổi state vector với server và hợp nhất hai phía. Vì Yjs là CRDT, việc hợp nhất các thay đổi ngoại tuyến với những thay đổi đã xảy ra trên server trong lúc đó luôn cho kết quả nhất quán, không mất dữ liệu và không cần người dùng can thiệp.

2.8. Tự động tạo phiên bản (auto-versioning)

Để người dùng có thể xem lại và khôi phục lịch sử, hệ thống tự động tạo các snapshot. `scheduleAutoSnapshot` áp dụng chiến lược kết hợp hai điều kiện:

- **Theo trạng thái nhàn rỗi (idle):** chốt một bản sau 2 phút kể từ khi người dùng ngừng gõ.
- **Theo thời gian tối đa (maxWait):** nếu người dùng gõ liên tục không nghỉ, vẫn chốt một bản checkpoint sau tối đa 10 phút.
- **Chốt khi đóng:** thêm một bản cuối khi người dùng cuối rời tài liệu (`flushSnapshotOnStore`).

Để tránh tạo các bản trùng nhau, mỗi snapshot được so sánh bằng băm SHA-1 của trạng thái: nếu nội dung không đổi so với bản gần nhất thì bỏ qua. Ngoài ra, hàm `pruneAutoVersions` tỉa thưa dần các bản tự động để danh sách gọn gàng: giữ tất cả bản trong 1 giờ gần nhất, giữ một bản mỗi giờ với khoảng 1–24 giờ, và một bản mỗi ngày với phần cũ hơn. Các bản người dùng lưu thủ công (`isAuto = false`, có đặt tên) không bao giờ bị tỉa.

```
export function scheduleAutoSnapshot(documentId, userId, state) {
  lastEditorByDoc.set(documentId, userId)
  pendingSnapshotState.set(documentId, state)
  if (!dirtySince.has(documentId)) dirtySince.set(documentId, Date.now())
  const since = dirtySince.get(documentId)
  if (Date.now() - since >= AUTO_MAX_WAIT_MS) // gõ liên tục > 10'
    return void maybeAutoSnapshot(documentId)
  // ngược lại: chốt sau khi ngừng gõ 2 phút (idle)
  clearTimeout(snapshotTimers.get(documentId))
  snapshotTimers.set(documentId,
    setTimeout(() => maybeAutoSnapshot(documentId), AUTO_IDLE_MS))
}
```

2.9. Tính nhất quán cuối và các đảm bảo của hệ thống

Mô hình đồng bộ của collab-editor mang lại tính nhất quán cuối (eventual consistency): tại một thời điểm các bản sao có thể tạm khác nhau do độ trễ, nhưng khi mọi update đã được trao đổi xong, tất cả chắc chắn hội tụ về cùng một trạng thái. Tổng hợp lại, hệ thống đảm bảo:

- **Hội tụ mạnh (strong eventual consistency):** hai bản sao nhận cùng tập update sẽ có trạng thái giống hệt nhau, nhờ bản chất CRDT của Yjs.
- **Không mất dữ liệu:** thay đổi được giữ ở CRDT cục bộ + IndexedDB (client) và Redis + PostgreSQL (server); Redis có thể lỗi mà không ảnh hưởng tính bền vững.
- **Độ trễ thấp:** thao tác áp dụng ngay cục bộ rồi mới đồng bộ; đường lưu trữ được tách khỏi đường phát update.
- **Chịu lỗi và ngoại tuyến:** mất mạng hay sự cố Redis đều được xử lý bằng fallback và đồng bộ lại khi phục hồi.

Nhờ kết hợp CRDT (Yjs) cho hòa giải xung đột, WebSocket (Hocuspocus) cho truyền tải thời gian thực và lưu trữ hai tầng (Redis + PostgreSQL) cho tốc độ lẫn độ bền, collab-editor đáp ứng trọn vẹn bốn mục tiêu thiết kế đặt ra ở mục 1.1.

III. GIAO THỨC TRAO ĐỔI DỮ LIỆU

Để tối ưu hóa hiệu năng, giảm thiểu băng thông đường truyền và đảm bảo trải nghiệm tương tác thời gian thực mượt mà cho người soạn thảo, hệ thống áp dụng mô hình giao thức lai. Cấu trúc trao đổi dữ liệu được phân tách một cách rõ ràng thành hai nhánh giao tiếp mạng độc lập:

- **Nhánh giao tiếp tĩnh** : Sử dụng mô hình *Yêu cầu - Phản hồi (Request-Response)* để quản lý các tác vụ cấu hình hệ thống tĩnh và phân quyền người dùng qua giao thức HTTP kết hợp RESTful API.
- **Nhánh giao tiếp động** : Sử dụng kết nối *song công (Full-duplex)* liên tục qua **WebSockets** kết hợp giao thức giải quyết xung đột nhị phân **Yjs Sync Protocol** và giao thức trạng thái hiện diện **Yjs Awareness Protocol** để đồng bộ hóa hoạt động thời gian thực.

3.1. Giao thức HTTP kết hợp RESTful API

3.1.1. Giới thiệu và định nghĩa:

- **HTTP (Hypertext Transfer Protocol)** là giao thức truyền tải siêu văn bản hoạt động theo mô hình Yêu cầu - Phản hồi (Request-Response) ngắn hạn trên nền TCP.
- **REST (Representational State Transfer)** là một phong cách kiến trúc thiết kế phần mềm định hướng tài nguyên (Resource-oriented). REST tận dụng trực tiếp các phương thức truyền tin sẵn có của HTTP để thực hiện các thao tác quản trị dữ liệu một cách chuẩn hóa.

3.1.2 Cách thức áp dụng trong dự án:

Được sử dụng cho toàn bộ các chức năng quản trị hệ thống tĩnh bao gồm: Đăng ký, đăng nhập người dùng; tải danh sách tài liệu cá nhân/chia sẻ/gần đây/yêu thích; thiết lập quyền hạn chia sẻ tài liệu; ghi nhận lịch sử mở tài liệu (/view) và truy xuất/khôi phục danh sách các phiên bản lịch sử của tài liệu.

3.1.3. Định dạng dữ liệu và bảo mật:

- **Định dạng dữ liệu**: Sử dụng chuỗi định dạng **JSON (application/json)** chuẩn hóa cho cả Request và Response Body, giúp việc truyền tải dữ liệu có cấu trúc trở nên rõ ràng và dễ dàng chuyển đổi sang đối tượng ngôn ngữ lập trình.
- **Bảo mật**: Xác thực phi trạng thái (Stateless) bằng mã thông báo **JWT (JSON Web Token)**. Token được phát hành sau khi đăng nhập thành công và bắt buộc đính kèm vào HTTP Header dưới dạng Authorization: Bearer <token> để kiểm tra quyền hạn của người dùng trên mỗi yêu cầu gửi lên máy chủ.

3.1.4. Lý do lựa chọn sử dụng:

Đối với các tác vụ không đòi hỏi cập nhật liên tục từng mili-giây như đăng nhập hoặc liệt kê file, việc sử dụng HTTP/REST giúp hệ thống hoạt động cực kỳ ổn định, bảo mật và dễ dàng lưu trữ bộ nhớ đệm (Caching). Việc thiết kế API theo chuẩn RESTful giúp mã nguồn của cả Client và Server có tính nhất quán cao, dễ dàng mở rộng và bảo trì.

3.1.5. Đánh giá ưu và nhược điểm:

- **Ưu điểm:** Đơn giản, dễ kiểm thử, bảo mật tốt với cơ chế JWT, hoạt động phi trạng thái (Stateless) giúp máy chủ dễ dàng mở rộng quy mô (Scale-out) mà không tốn bộ nhớ lưu trữ phiên làm việc của người dùng.
- **Nhược điểm:** Không hỗ trợ cập nhật dữ liệu chủ động từ phía Server đến Client. Do đó không thể áp dụng cho việc biên tập đồng thời thời gian thực (nếu dùng HTTP để đồng bộ chữ gõ, máy chủ sẽ bị quá tải request ngay lập tức).

3.2. Giao thức Websockets

3.2.1. Giới thiệu và định nghĩa:

WebSockets là giao thức truyền thông hai chiều (song công - full-duplex) qua một kết nối TCP duy nhất. Khác với HTTP hoạt động theo cơ chế Client hỏi - Server mới trả lời, WebSockets giữ kết nối luôn mở, cho phép cả Client và Server chủ động đẩy dữ liệu sang nhau bất kỳ lúc nào mà không cần bắt tay kết nối lại.

3.2.2. Cách thức áp dụng trong dự án:

WebSockets được sử dụng làm nền tảng cho tính năng biên tập tài liệu cộng tác thời gian thực. Mỗi khi người dùng mở một tài liệu, client React sẽ mở một kết nối Socket đến server Hocuspocus để liên tục đồng bộ phím gõ và tọa độ con trỏ chuột của toàn bộ thành viên đang online.

3.2.3. Định dạng dữ liệu và bảo mật:

- **Định dạng dữ liệu:** Dữ liệu truyền tải qua WebSocket trong dự án được mã hóa dưới dạng dòng **nhị phân (Binary - Uint8Array)** thông qua thư viện Yjs để tối ưu hóa tối đa băng thông đường truyền.
- **Bảo mật:** Xác thực bằng **JWT Token** ngay trong bước bắt tay thiết lập kết nối (onAuthenticate). Server sẽ giải mã token để xác định ID người dùng và kiểm tra quyền đọc/ghi của người dùng trên tài liệu đó trước khi chấp thuận duy trì kết nối.

3.2.4. Lý do lựa chọn sử dụng:

Biên tập tài liệu thời gian thực đòi hỏi độ trễ cực thấp (dưới 50ms) để người dùng có cảm giác như đang gõ trực tiếp trên máy cục bộ. Nếu dùng HTTP API truyền thông (cứ gõ

một chữ lại gửi 1 request HTTP) thì máy chủ sẽ lập tức bị quá tải (DDoS) và độ trễ sẽ cực kỳ cao do chi phí thiết lập kết nối liên tục.

3.2.5. Đánh giá ưu và nhược điểm:

- **Ưu điểm:** Độ trễ cực thấp, tiết kiệm tài nguyên máy chủ và băng thông đường truyền nhờ duy trì kết nối đơn lẻ và truyền tải dữ liệu nhị phân siêu nhẹ.
- **Nhược điểm:** Đòi hỏi máy chủ phải duy trì trạng thái kết nối liên tục (Stateful), tốn bộ nhớ RAM để quản lý các kết nối mở và cần cơ chế xử lý kết nối lại (reconnection) phức tạp khi mạng của client không ổn định.

3.3 Giao thức Yjs Sync Protocol

3.3.1. Giới thiệu và định nghĩa:

Yjs Sync Protocol là giao thức đồng bộ hóa dữ liệu tăng ứng dụng hoạt động dựa trên thuật toán CRDT (Conflict-free Replicated Data Type). Giao thức này định nghĩa quy tắc trao đổi các khối dữ liệu nhị phân chứa thông tin thay đổi văn bản để đảm bảo các bản sao tài liệu phân tán ở các máy khách khác nhau tự động hợp nhất về cùng một kết quả duy nhất mà không xảy ra xung đột dữ liệu.

3.3.2. Cách thức áp dụng trong dự án:

Áp dụng trực tiếp tại màn hình biên tập soạn thảo tài liệu cộng tác (EditorPage). Giao thức này đóng gói toàn bộ thao tác thêm, xóa ký tự của người dùng thành các khối thay đổi (Delta Updates) và truyền qua kênh WebSocket để đồng bộ tức thời giữa các trình duyệt của người dùng khác nhau và máy chủ.

3.3.3. Định dạng dữ liệu và bảo mật:

- **Định dạng dữ liệu:** Mã hóa hoàn toàn dưới dạng dòng nhị phân siêu gọn nhẹ (Binary - Uint8Array) thông qua cơ chế nén của Yjs (sử dụng hàm `Y.encodeStateAsUpdate`).
- **Bảo mật:** Tích hợp chặt chẽ với phân quyền của hệ thống. Server kiểm tra quyền chỉnh sửa động thông qua sự kiện `beforeHandleMessage` trên từng gói tin đồng bộ nhị phân gửi lên. Nếu người dùng bị hạ quyền xuống chỉ xem (Viewer), hệ thống sẽ lập tức chặn các thông điệp đồng bộ sửa đổi của người dùng đó.

3.3.4. Lý do lựa chọn sử dụng:

Khi cộng tác nhiều người, việc gõ chữ xảy ra liên tục (mỗi giây có hàng chục phím bấm từ nhiều người khác nhau). Yjs Sync Protocol giúp giải quyết bài toán xung đột mà các hệ thống cũ gặp phải (người này đè chữ lên người kia) bằng thuật toán CRDT toán học thông minh. Đồng thời, việc mã hóa nhị phân giúp dung lượng gói tin cực nhỏ, bảo vệ đường truyền mạng không bị nghẽn.

3.3.5. Đánh giá ưu và nhược điểm:

- **Ưu điểm:** Tự động giải quyết xung đột 100% không cần máy chủ phân xử trung tâm; dung lượng truyền tải nhị phân siêu nhẹ; hỗ trợ đồng bộ hóa ngoại tuyến (offline-first) hoàn hảo nhờ cơ chế lưu trữ IndexedDB cục bộ.
- **Nhược điểm:** Đòi hỏi chi phí tính toán CPU trên trình duyệt client cao hơn để tính toán thuật toán CRDT; cấu trúc dữ liệu nhị phân phức tạp, khó gỡ lỗi (debug) hơn so với định dạng văn bản JSON thông thường.

3.4 Giao thức Yjs Awareness Protocol

3.4.1. Giới thiệu và định nghĩa:

Yjs Awareness Protocol là giao thức tầng ứng dụng dùng để quản lý và chia sẻ trạng thái hiện diện tạm thời (Presence/Ephemeral state) của các máy khách trong một hệ thống cộng tác phân tán.

3.4.2. Cách thức áp dụng trong dự án:

Được sử dụng cho hai tính năng trực quan chính:

- **Hiện thị danh sách ảnh đại diện (avatar)** của tất cả các thành viên hiện đang cùng mở và online trong cùng một tài liệu soạn thảo.
- **Hiện thị thời gian thực vị trí con trỏ chuột, vùng bôi đen (selection), tên hiển thị và màu sắc định danh** của từng người soạn thảo khi họ di chuyển chuột trên văn bản.

3.4.3. Định dạng dữ liệu và bảo mật:

- **Định dạng dữ liệu:** Mã hóa dưới dạng trạng thái cặp key-value tạm thời trong bộ nhớ của các Client và phân phát nhanh chóng thông qua kết nối.
- **Bảo mật:** Chỉ truyền phát thông tin trạng thái giữa các client đã được xác thực JWT thành công trước đó thông qua đường truyền WebSocket. Bản tin Awareness mang tính chất **phi lưu trữ (ephemeral)**, hoàn toàn không được ghi nhận vào ổ đĩa vật lý hay cơ sở dữ liệu (PostgreSQL/Redis) để tránh gây nghẽn băng thông và ổ cứng máy chủ.

3.4.4. Lý do lựa chọn sử dụng:

Hiện thị con trỏ soạn thảo của người khác giúp tạo ra cảm giác "cộng tác tự nhiên" và tăng tính tương tác xã hội. Vì vị trí con trỏ thay đổi liên tục theo mỗi milimet di chuột (lên tới hàng nghìn gói tin mỗi phút), việc sử dụng một giao thức phi lưu trữ, chỉ truyền phát qua bộ nhớ (in-memory broadcast) như Yjs Awareness là giải pháp tối ưu duy nhất tránh gây sập hệ thống cơ sở dữ liệu.

3.4.5. Đánh giá ưu và nhược điểm:

- **Ưu điểm:** Tốc độ truyền tải siêu tốc, không tạo gánh nặng lưu trữ cho máy chủ và cơ sở dữ liệu, tự động dọn dẹp (auto-cleanup) thông tin người dùng ngay khi họ đóng tab hoặc mất mạng.
- **Nhược điểm:** Do tính chất tạm thời, nếu người dùng bị mất kết nối mạng đột ngột (chưa kịp timeout kết nối), con trỏ soạn thảo của họ trên màn hình của những người khác có thể bị đứng yên ("đóng băng") trong một khoảng thời gian ngắn trước khi cơ chế tự dọn dẹp được kích hoạt.

IV. THỬ NGHIỆM VÀ ĐÁNH GIÁ

4.1. Giao diện trang đăng nhập, đăng kí

Collab Editor

Soạn thảo cộng tác realtime

Đăng nhập

Đăng ký

Email

example@email.com

Mật khẩu

.....

Đăng nhập

hoặc

 Đăng nhập bằng Google

Collab Editor

Soạn thảo cộng tác realtime

Đăng nhập

Đăng ký

Tên

Nguyễn Văn A

Email

example@email.com

Mật khẩu

.....

Xác nhận mật khẩu

.....

Đăng ký

hoặc



Đăng ký bằng Google

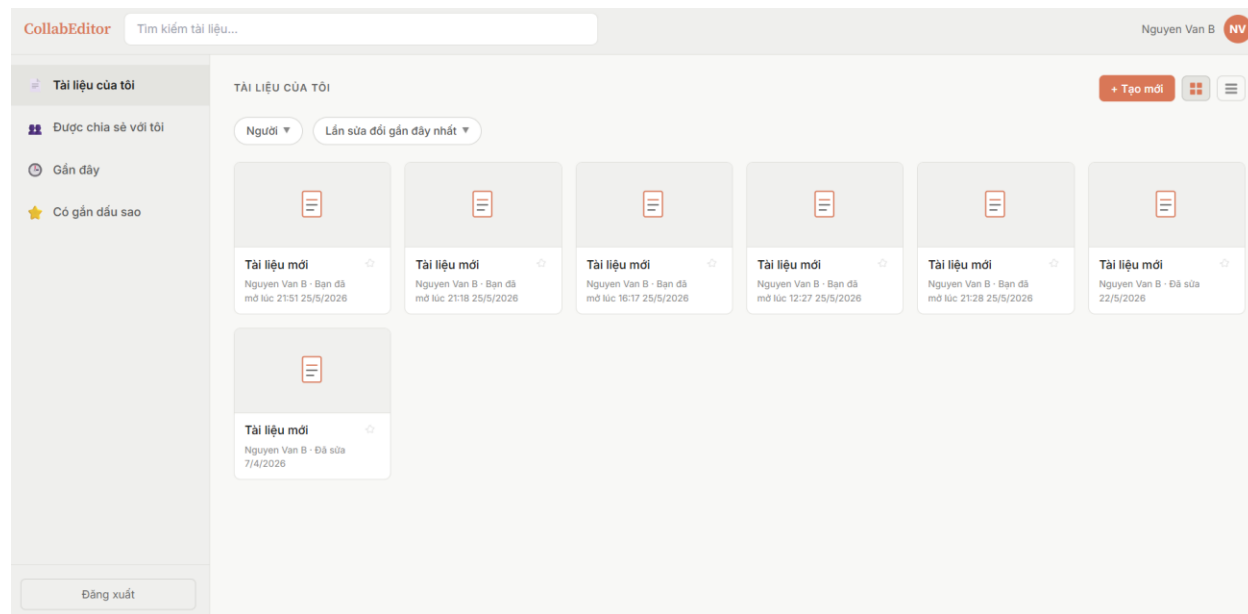
Kết quả thực nghiệm

Chức năng đăng ký và đăng nhập hoạt động đúng với luồng xác thực đã thiết kế. Hệ thống phát hành JWT sau khi đăng nhập thành công và lưu token vào localStorage thông qua Zustand store, đảm bảo phiên làm việc được duy trì khi tải lại trang.

Điểm đạt được:

- Giao diện trực quan, phân biệt rõ hai luồng đăng nhập và đăng ký.
- Validation phía client phát hiện các trường hợp thiếu thông tin, email sai định dạng và mật khẩu không khớp trước khi gửi request lên server.
- Thông báo lỗi rõ ràng khi sai mật khẩu, tài khoản không tồn tại hoặc mất kết nối (xử lý tại auth.controller.ts).
- Token JWT được đính kèm tự động vào mọi request tiếp theo thông qua Axios interceptor trong services/api.ts, và tự động đăng xuất khi nhận mã lỗi 401.

4.2. Giao diện trang chủ



Kết quả thử nghiệm

Sau khi đăng nhập, trang chủ hiển thị đầy đủ các tài liệu mà người dùng sở hữu hoặc được chia sẻ. Các tính năng lọc và tìm kiếm hoạt động chính xác.

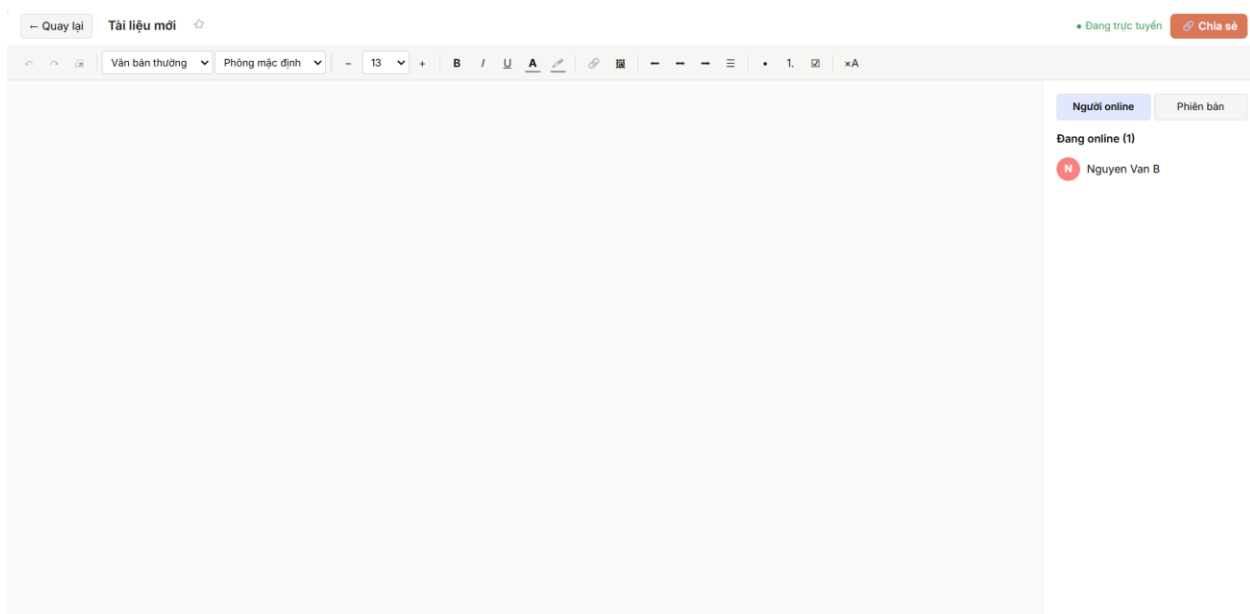
Điểm đạt được:

- Phân loại tài liệu rõ ràng: tài liệu của tôi, được chia sẻ, đánh dấu sao, gần đây — phù hợp với thói quen sử dụng thực tế.
- Tìm kiếm hỗ trợ cả theo tên tài liệu lẫn nội dung bên trong (qua trường contentPreview được cập nhật sau mỗi lần lưu), giúp người dùng không nhớ tên vẫn tìm được tài liệu.

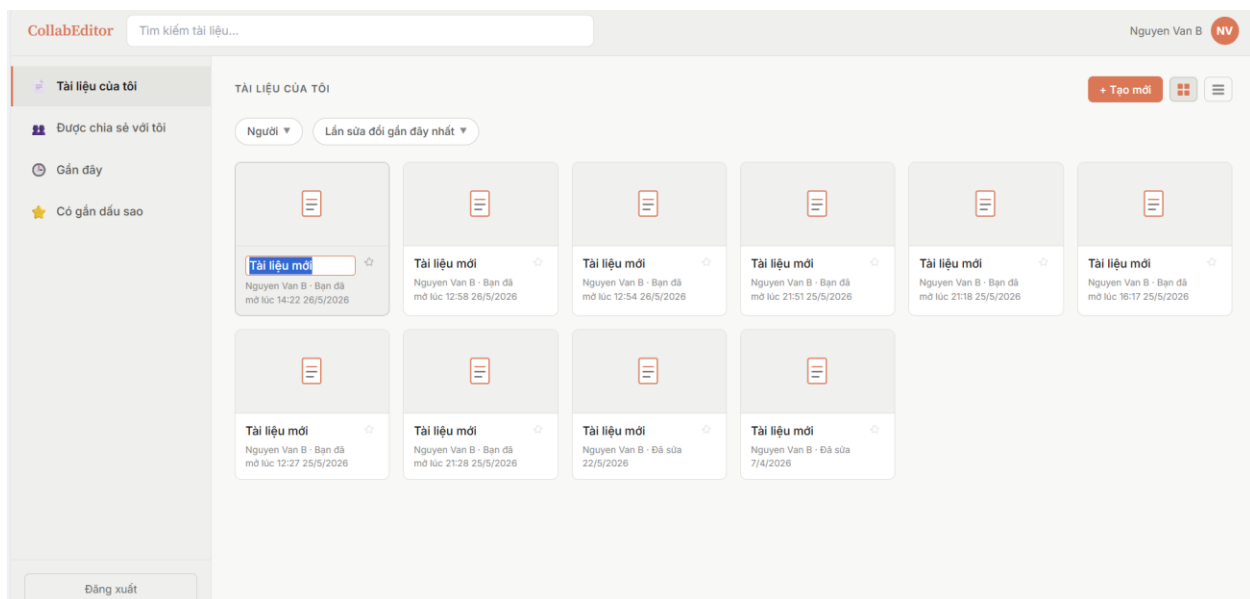
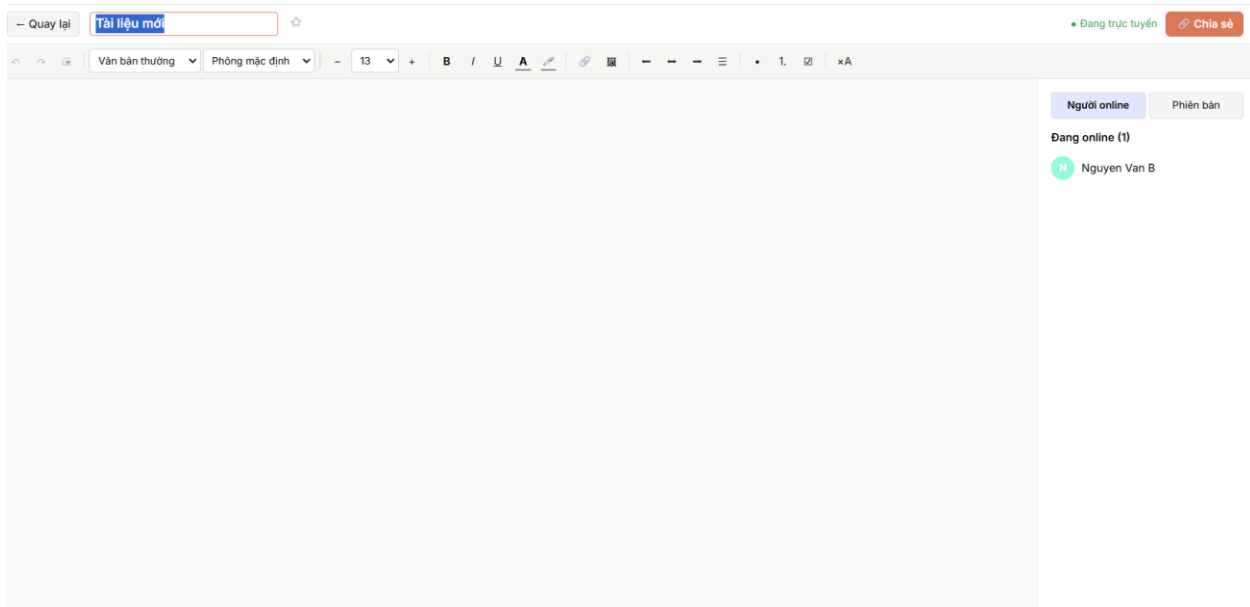
- Lịch sử xem (DocumentViewed) ghi lại thời điểm mở gần nhất, trang "Gần đây" phản ánh đúng hành vi duyệt của người dùng.
- Tính năng đánh dấu sao (StarredDocument) hoạt động nhất quán giữa các phiên.

4.3. Tạo và quản lý tài liệu

- Người dùng có thể tạo tài liệu bằng nút tạo mới
- Đây là giao diện bên trong 1 văn bản gồm các công cụ chỉnh sửa về Font chữ, cỡ chữ, căn lề... giúp người dùng có thể chỉnh sửa văn bản



- Người dùng đổi tên tài liệu ngay bên trong tài liệu hoặc ở trang chủ bên ngoài trang chủ



Kết quả thử nghiệm

Người dùng tạo tài liệu mới thành công bằng một thao tác nhấn nút. Tài liệu xuất hiện ngay trên dashboard và có thể mở để soạn thảo. Giao diện thanh công cụ (Toolbar) cung cấp đầy đủ định dạng cơ bản.

Điểm đạt được:

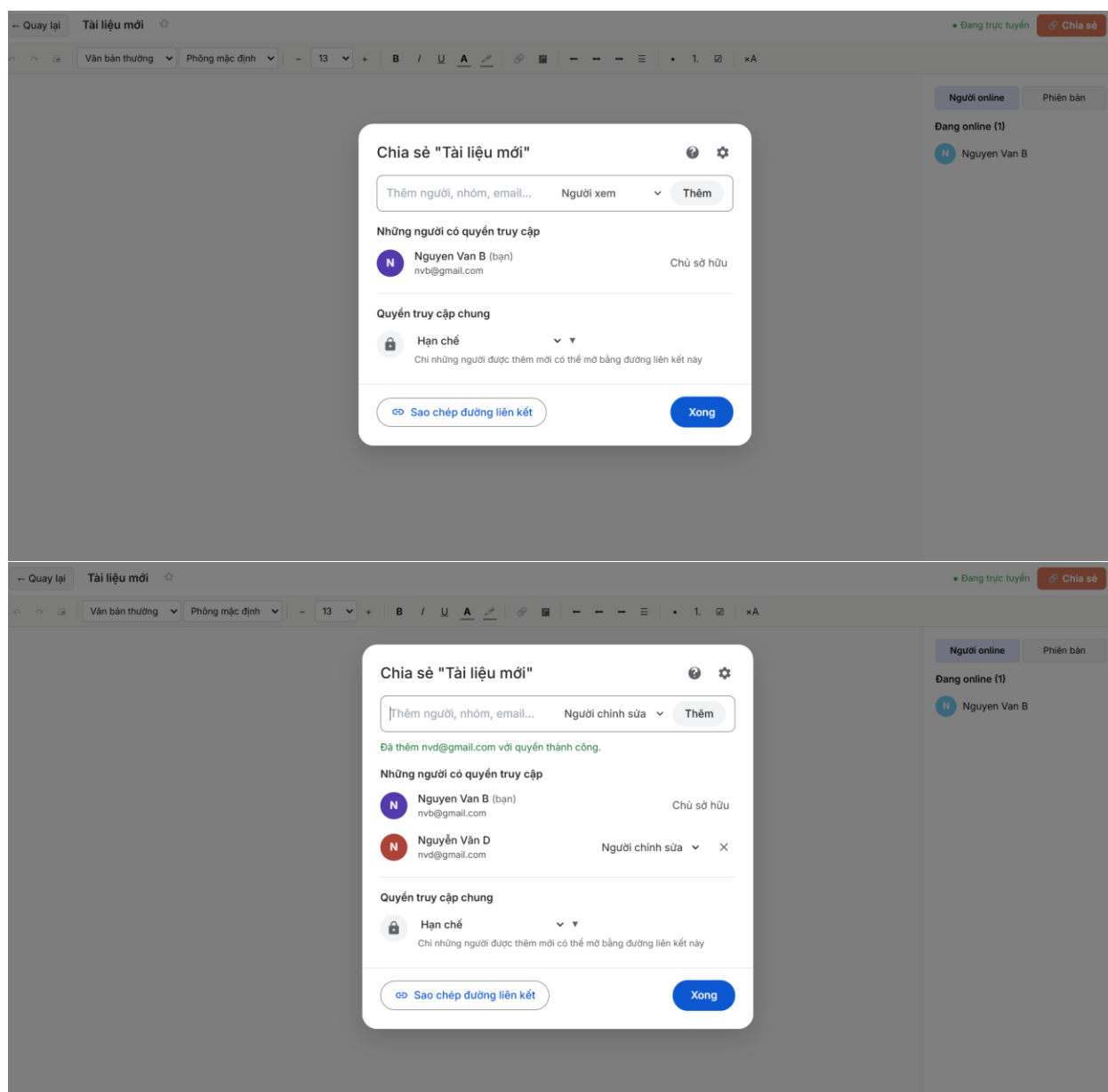
Toolbar hỗ trợ đa dạng định dạng: đậm, nghiêng, gạch chân, heading (H1–H3), danh sách có thứ tự và không có thứ tự, căn lề, cỡ chữ tùy chỉnh.

Đổi tên tài liệu được hỗ trợ tại hai nơi — ngay trong trình soạn thảo và từ trang chủ — giúp người dùng linh hoạt trong việc quản lý.

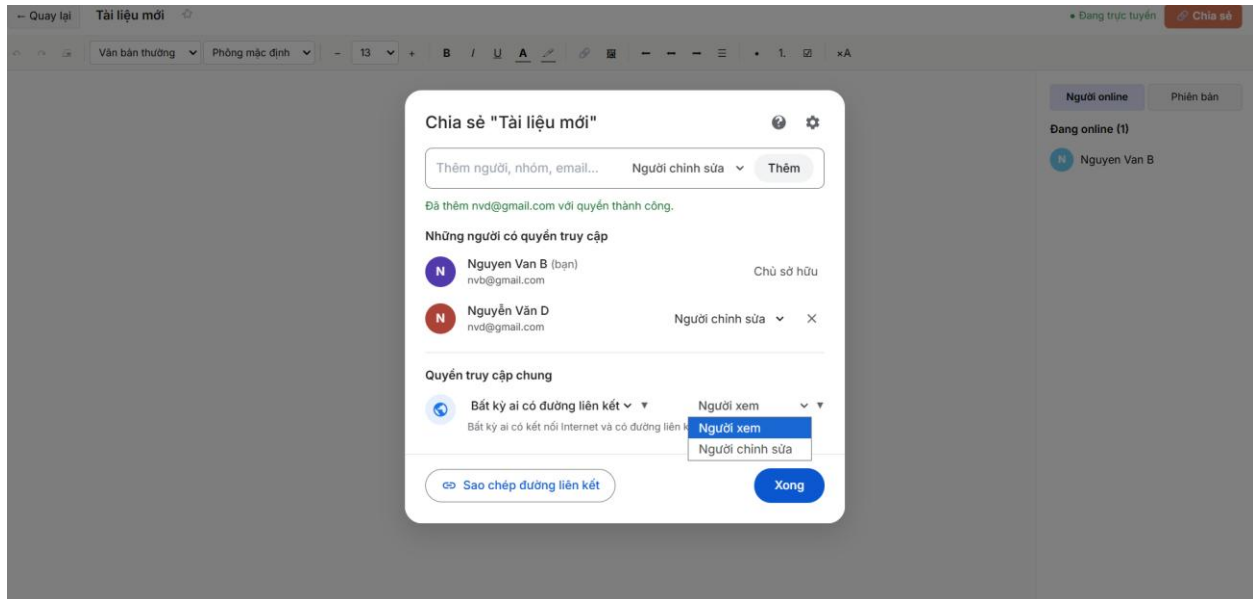
Thay đổi tên đồng bộ ngay lập tức qua REST API (PATCH /api/documents/:id), phản ánh kịp thời trên dashboard mà không cần tải lại trang.
Tiêu đề tài liệu mặc định rõ ràng, tránh tình trạng nhiều tài liệu không tên.

4.4. Chia sẻ tài liệu

- Để chỉnh sửa cộng tác ta cần thêm thành viên vào nhóm để cùng chỉnh sửa văn bản thông qua việc thêm thành viên bằng gmail hoặc chia sẻ đường link
- Chỉ chủ sở hữu mới có quyền chỉnh sửa quyền hạn hay thêm bớt thành viên



- Mở đường liên kết đến để share đến tất cả mọi người



Kết quả thử nghiệm

Chủ sở hữu chia sẻ tài liệu thành công qua hai phương thức: mời thành viên theo địa chỉ email và chia sẻ đường liên kết công khai. Phân quyền được kiểm soát chặt chẽ.

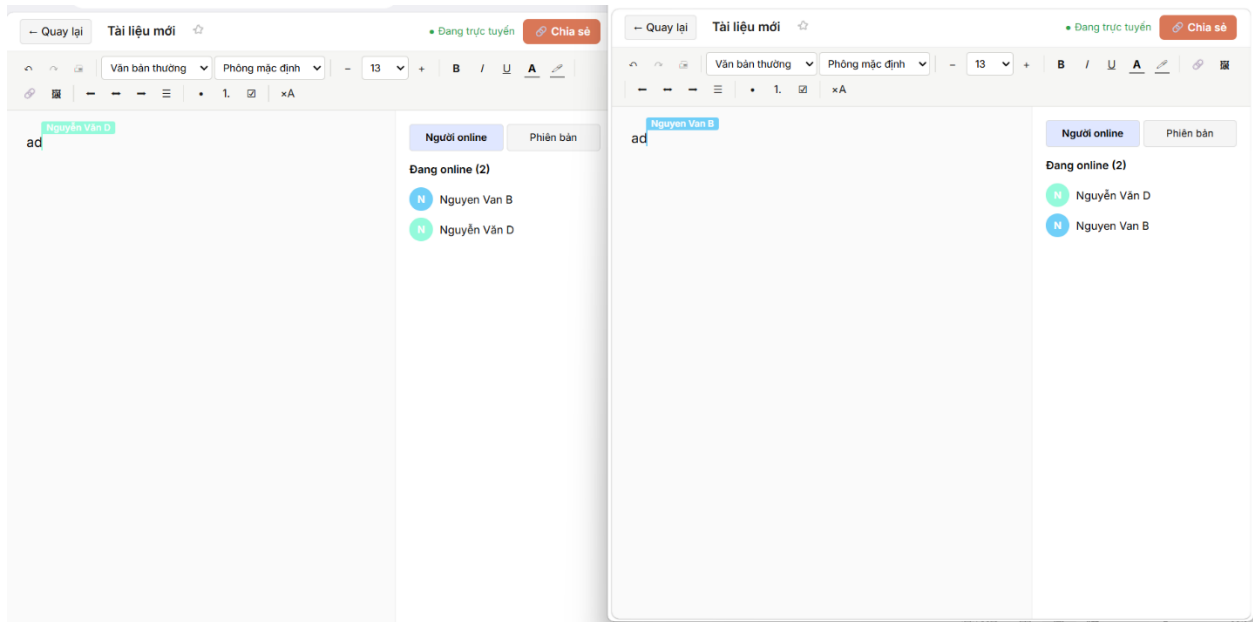
Điểm đạt được:

- Hệ thống RBAC ba cấp (OWNER / EDITOR / VIEWER) hoạt động nhất quán: VIEWER không thể gửi bản cập nhật CRDT lên server nhờ hook beforeHandleMessage tái kiểm tra trên từng tin nhắn WebSocket, không chỉ khi kết nối.
- Chỉ OWNER mới có quyền thêm/xóa thành viên và thay đổi quyền hạn — đảm bảo nguyên tắc least privilege.
- Phân quyền thu hồi tức thì: nếu OWNER hạ quyền một thành viên đang online, thay đổi có hiệu lực ngay mà không cần thành viên đó đóng và mở lại tài liệu.
- Hỗ trợ chia sẻ link công khai với quyền đọc hoặc chỉnh sửa, phù hợp cho trường hợp cộng tác không yêu cầu đăng ký tài khoản.

4.5. Quản lý người dùng đang chỉnh sửa

-Thành viên vào chỉnh sửa sẽ hiện trong danh sách người dùng online

-Khi người dùng chỉnh sửa trên văn bản sẽ hiện thị con trỏ của từng người dùng cho thành viên còn lại biết



Kết quả thử nghiệm

Khi hai phiên trình duyệt cùng mở một tài liệu, danh sách người dùng trực tuyến cập nhật ngay lập tức. Con trỏ của mỗi thành viên hiển thị đúng vị trí và được nhận dạng bằng màu riêng biệt.

Điểm đạt được

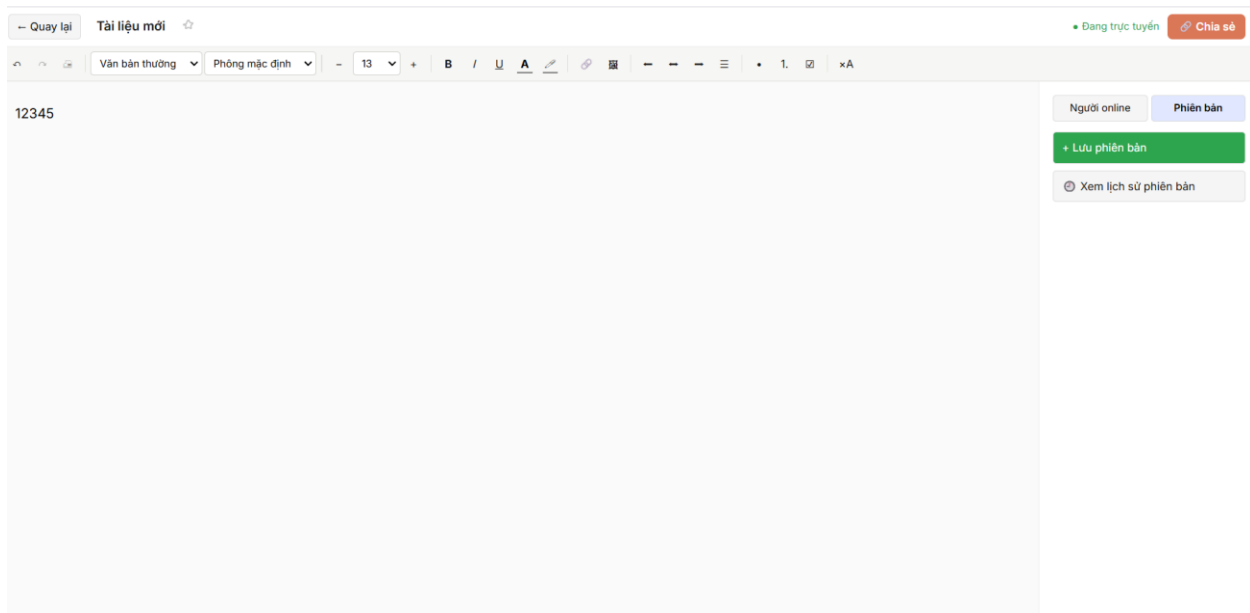
- Yjs Awareness Protocol truyền trạng thái con trỏ và vùng bôi đen hoàn toàn qua bộ nhớ (in-memory), không ghi vào cơ sở dữ liệu, đảm bảo tốc độ cập nhật gần như tức thì và không tạo áp lực lên Redis/PostgreSQL.
- Mỗi thành viên được gán màu nhận dạng nhất quán trong suốt phiên làm việc (component CollabCursor.tsx và UserList.tsx).
- Danh sách người dùng tự động dọn sạch khi thành viên đóng tab hoặc mạng timeout, không để lại "bóng ma" trong danh sách.
- Tính năng hoạt động mượt mà ngay cả khi nhiều người cùng gõ đồng thời.

4.6. Lịch sử chỉnh sửa và phiên bản

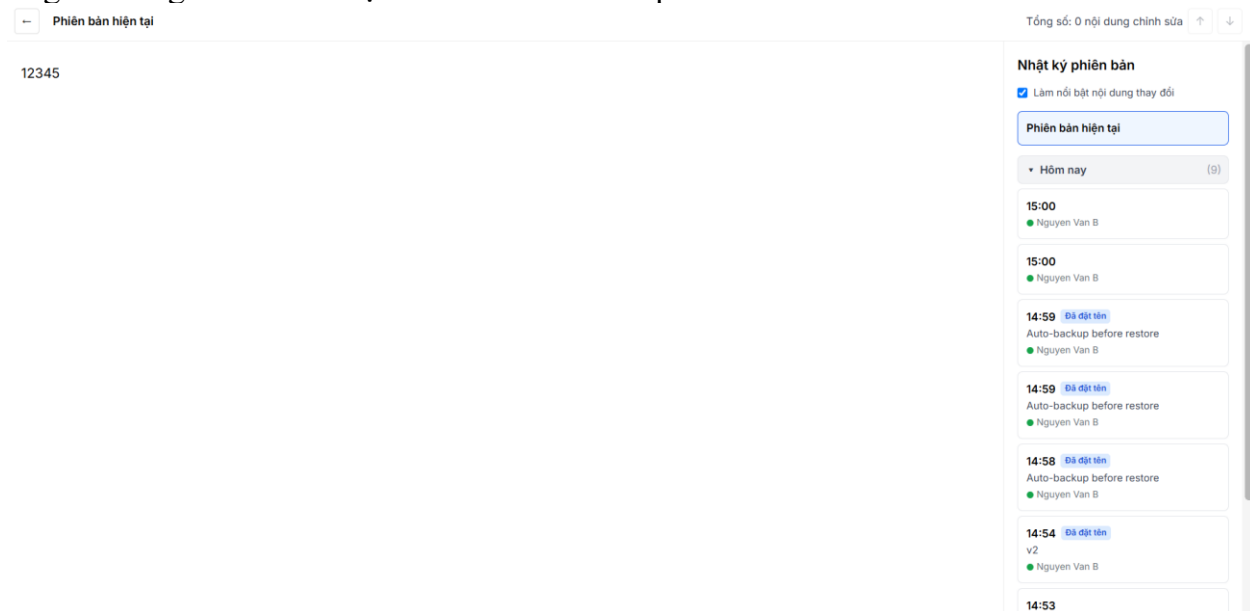
-Version được lưu tự động hoặc là lưu thủ công

-Người dùng có thể xem lịch sử chỉnh sửa của phiên bản

-



-Người dùng có thể xem lịch sử chỉnh sửa của phiên bản



← 15:00:28 26/5/2026

Tổng số: 1 nội dung chỉnh sửa ↑ ↓ Khôi phục phiên bản này

add

Nhật ký phiên bản

☒ Làm nổi bật nội dung thay đổi

Phiên bản hiện tại

Hôm nay (9)

15:00

● Nguyen Van B

15:00

● Nguyen Van B

14:59 Đã đặt tên

Auto-backup before restore

● Nguyen Van B

14:59 Đã đặt tên

Auto-backup before restore

● Nguyen Van B

14:58 Đã đặt tên

Auto-backup before restore

● Nguyen Van B

14:54 Đã đặt tên

v2

● Nguyen Van B

14:53

-Có thể khôi phục phiên bản

← 15:00:28 26/5/2026

localhost:5173 cho biết
Khôi phục về bản v45? Hệ thống sẽ tự lưu bản hiện tại trước.

OK

Hủy

Tổng số: 1 nội dung chỉnh sửa ↑ ↓ Khôi phục phiên bản này

add

Nhật ký phiên bản

☒ Làm nổi bật nội dung thay đổi

Phiên bản hiện tại

Hôm nay (9)

15:00

● Nguyen Van B

15:00

● Nguyen Van B

14:59 Đã đặt tên

Auto-backup before restore

● Nguyen Van B

14:59 Đã đặt tên

Auto-backup before restore

● Nguyen Van B

14:58 Đã đặt tên

Auto-backup before restore

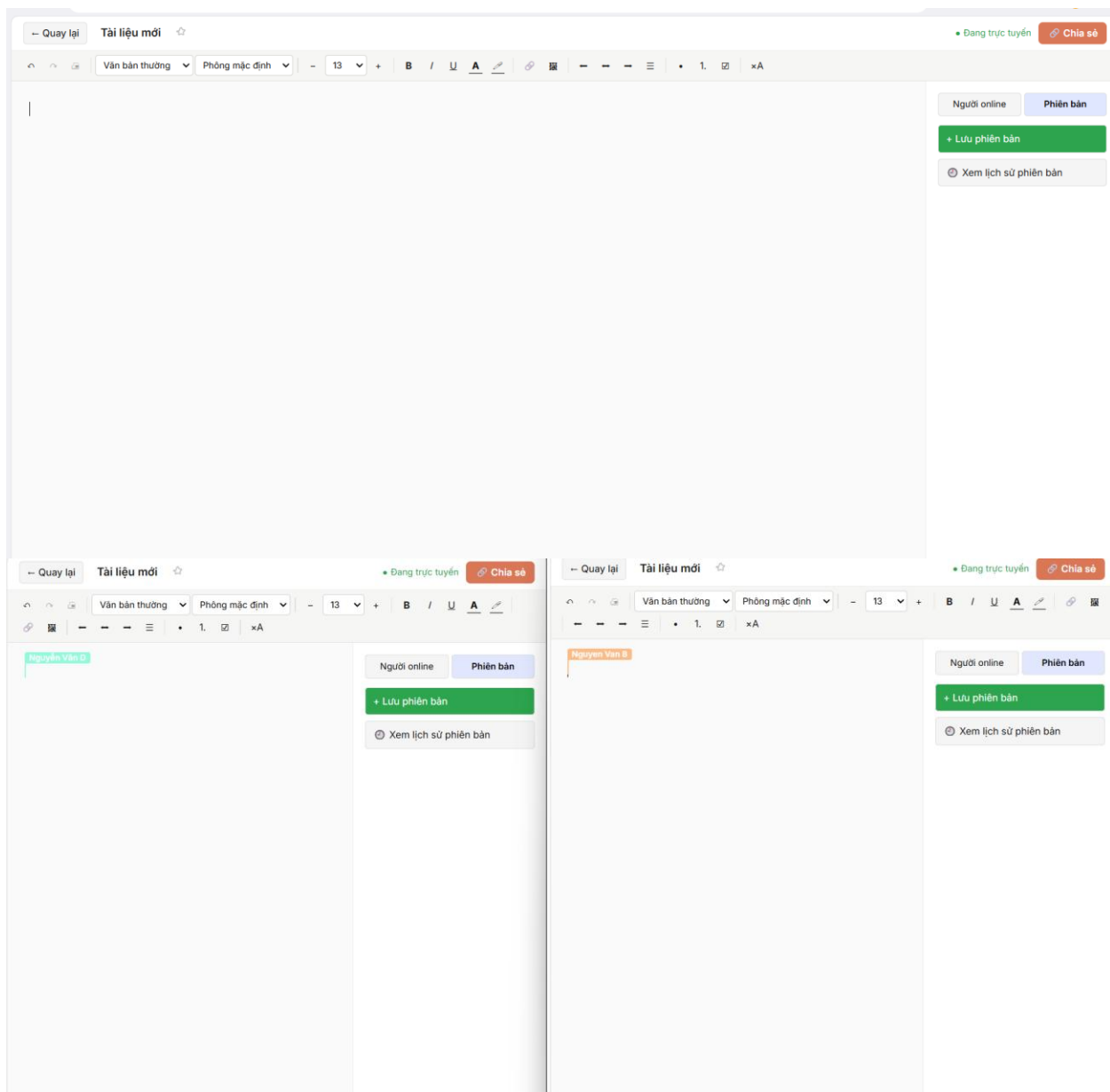
● Nguyen Van B

14:54 Đã đặt tên

v2

● Nguyen Van B

14:53



Kết quả thử nghiệm

Hệ thống tự động tạo phiên bản sau mỗi khoảng thời gian nhàn rỗi và khi người dùng cuối cùng rời tài liệu. Người dùng xem lại, so sánh và khôi phục các phiên bản thành công.

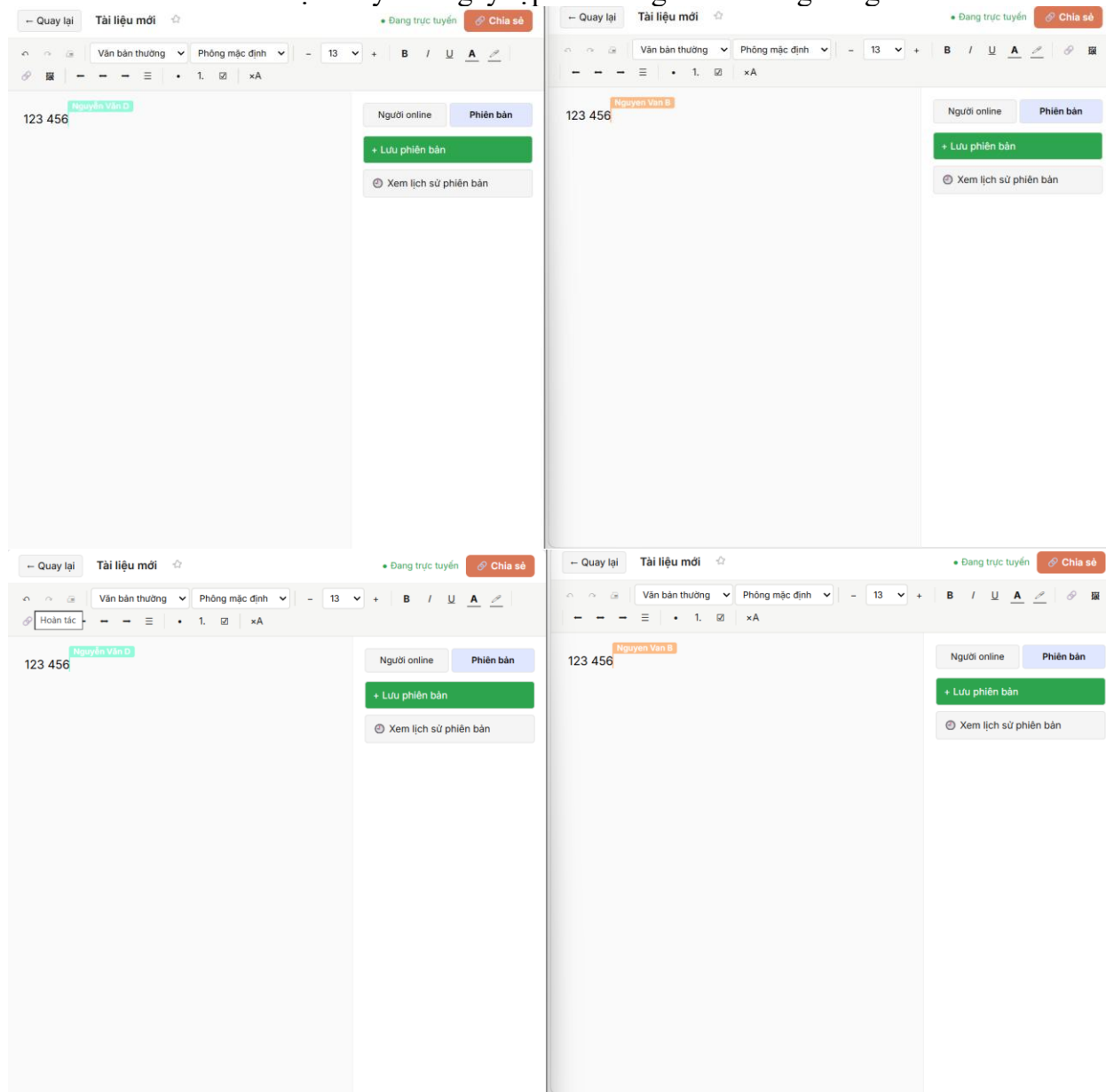
Điểm đạt được

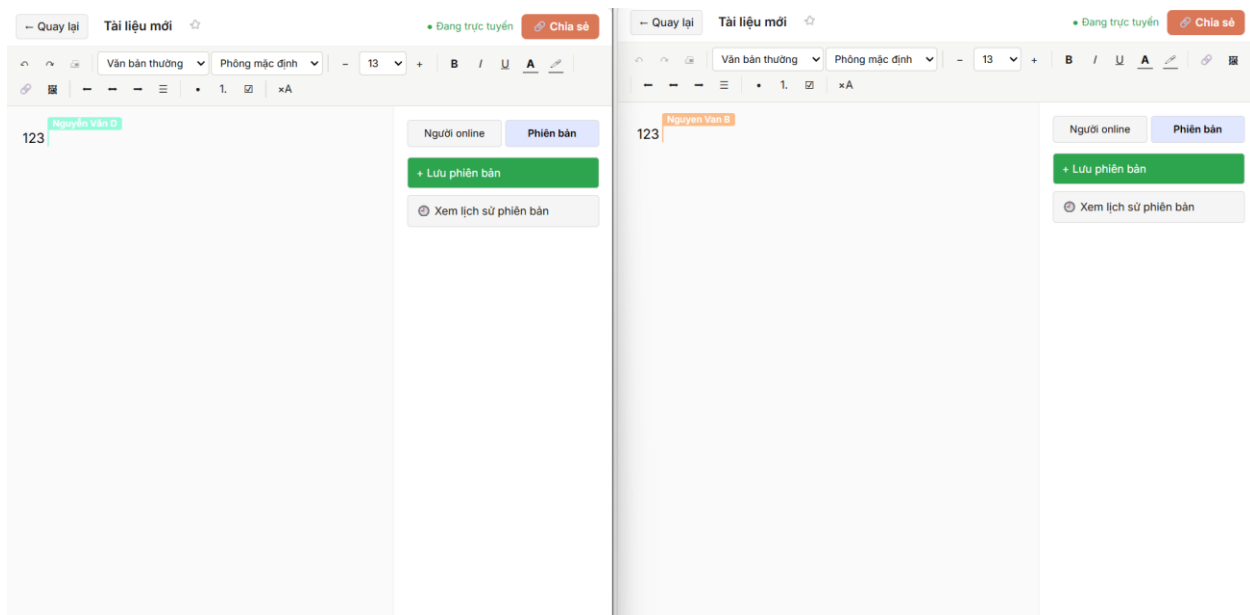
- Chiến lược hai điều kiện (idle 2 phút + tối đa 10 phút) cân bằng tốt giữa tần suất snapshot và tải hệ thống: tránh tạo quá nhiều bản ghi khi người dùng gõ liên tục, đồng thời đảm bảo không mất quá nhiều công sức nếu xảy ra sự cố.
- Cơ chế so sánh SHA-1 loại bỏ các bản snapshot trùng nội dung — tiết kiệm đáng kể không gian lưu trữ.

- Tỉa thừa thông minh (pruneAutoVersions): giữ đầy đủ bản gần nhất, gộp thừa dần theo thời gian.
- Bản lưu thủ công (có đặt tên, isAuto = false) không bao giờ bị tỉa, đảm bảo mốc quan trọng của người dùng luôn được giữ lại.
- Hỗ trợ khôi phục phiên bản bất kỳ về trạng thái hiện tại của tài liệu.

4.7. Undo/redo trong môi trường cộng tác

-Thao tác undo/redo được thay đổi ngay lập tức trong môi trường cộng tác





Kết quả thử nghiệm

Thao tác Undo/Redo hoạt động chính xác trong phiên cộng tác nhiều người: mỗi người chỉ hoàn tác được thao tác của chính mình, không ảnh hưởng đến chỉnh sửa của người khác.

Điểm đạt được

- Yjs quản lý lịch sử Undo/Redo riêng biệt cho từng clientID, khắc phục hoàn toàn vấn đề cổ điển của Operational Transformation khi Undo có thể "cuộn ngược" cả thay đổi của người khác.
- Tiptap's StarterKit được cấu hình tắt module history mặc định, tránh xung đột với Yjs UndoManager — đây là điểm kỹ thuật quan trọng được xử lý đúng.
- Phản hồi Undo/Redo tức thì (áp dụng cục bộ trước, đồng bộ sau), người dùng không cảm nhận độ trễ.
- Hoạt động nhất quán khi có thêm/bớt thành viên trong phiên, không gây ra trạng thái bất định.